LoRA and QLoRA Fine-Tuning

Adapting LLMs on a Limited Budget

Dr. Papa-Séga WADE

AIMS Senegal | Module 2 - Session 7

MODULE 2 - Session 7: efficient adaptation with LoRA and QLoRA

"Fine-tune behavior, retrieve knowledge"

Where We Stopped in Session 6

Recap

RAG plugs the LLM into your documents and gives it new **knowledge**. But the model's **behavior, style, and output format** are still those of the base model. Sometimes that is not enough.

We are here

Baseline Objective

By the end of this session

You will understand **when** to fine-tune (vs prompt or retrieve), **how** LoRA and QLoRA make it cheap, and **how to prepare data** for it, including synthetic data for African languages.

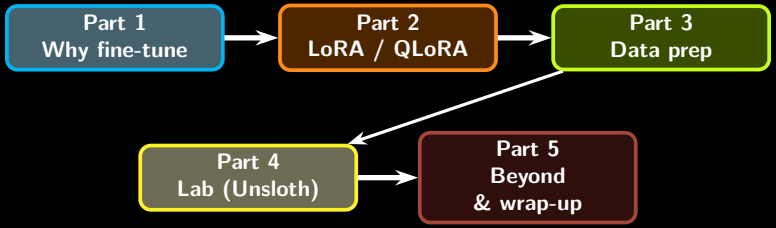
You will be able to

- explain why full fine-tuning is impractical,
- describe LoRA and QLoRA in 2 sentences,
- write a clean instruction-response dataset,
- run a LoRA fine-tune with Unsloth on Colab.

Guiding question

How would you adapt a 1B model to answer student questions in a fixed AIMS-Senegal style, using only a free Colab GPU?

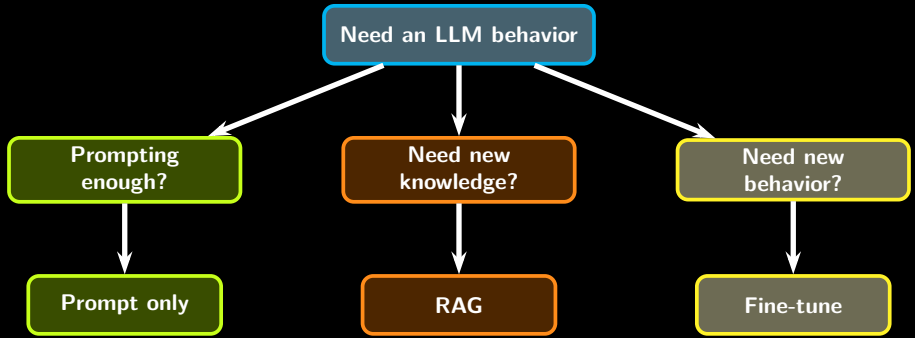
Session Roadmap



Core message

Fine-tune to change **behavior**. Use the cheapest trick that works, and always start with the smallest model that meets the bar.

The Decision Tree, Revisited



Heuristic

Fine-tune only when prompting and RAG cannot reach the bar. It costs more and ages faster than a prompt change.

Concrete Cases Where Fine-Tuning Helps

Style and format

- AIMS academic tone in French, every time,
- strict JSON output for downstream parsing,
- Wolof reformulation of medical advice.

Domain behavior

- triage agent that follows a Senegalese health protocol,
- code agent that uses internal AIMS-IT conventions.

Cases where fine-tuning is the wrong tool

- the model needs to **know** new facts → RAG,
- one-off tasks: just write a better prompt,
- rapidly changing data (regulations) → RAG,
- missing data quality → fix data first.

Why Full Fine-Tuning Is Impractical

The cost of training every weight

For a model with N parameters and Adam optimizer:

$$\text{VRAM} \approx (\mathbf{4}\,W + \mathbf{4}\,G + \mathbf{8}\,O) \cdot N \approx 16N \text{ bytes}$$

weights, gradients, optimizer states (m and v).

For a 7B model

$16 \times 7 \cdot 10^9 \approx 112$ GB VRAM. Out of reach for a Colab T4 (16 GB) or even an A100 (40-80 GB) for larger models.

What we actually want

Change a few **percent** of the model's behavior, not 100%. Updating 100% of parameters to do that is wasteful.

The lever

If most layers are already good, can we freeze them and learn only a small adjustment? **Yes: LoRA**.

LoRA: The Core Idea

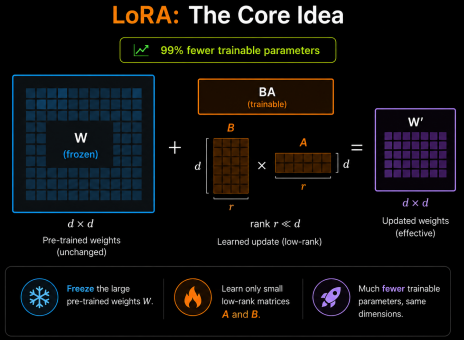
Definition

Freeze the pretrained weight matrix $W \in \mathbb{R}^{d \times d}$. Add a **low-rank update** $\Delta W = BA$ where $A \in \mathbb{R}^{r \times d}$, $B \in \mathbb{R}^{d \times r}$, and $r \ll d$.

$$W' = W + \alpha\, BA$$

Why it works

Empirically, the useful update is approximately low-rank. We do not need the full $d \times d$ degrees of freedom.



LoRA: Parameter Count Intuition

Counting trainable parameters

Full update on one $d \times d$ matrix: d^2 parameters.
LoRA update with rank r: A has rd, B has dr, total $2rd$.

$$\text{ratio} = \frac{2rd}{d^2} = \frac{2r}{d}$$

Small Qwen3-style example

If a layer has hidden size $d \approx 1024$ and rank $r = 8$, the ratio is $16/1024 \approx 1.56\%$. Combined with freezing all the rest, you train roughly **0.1 to 1%** of the model.

Practical hyperparameters

Typical: $r \in \{8, 16, 32\}$, $\alpha \in \{16, 32, 64\}$, dropout $\in \{0.05, 0.1\}$. Apply LoRA to attention projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and often MLP layers.

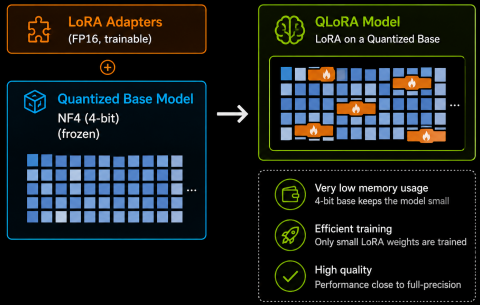
QLoRA: LoRA on a Quantized Base

The recipe

1. Load the base model in **4-bit (NF4)** quantization.
2. Freeze it.
3. Attach LoRA adapters (in FP16) to the relevant layers.
4. Train only the adapters.

Why this is a breakthrough

A 70B model fits on a **single 24 GB GPU** (4-bit weights = 35 GB $\rightarrow$ a few extra tricks bring it under 24 GB).



LoRA vs QLoRA in One Table

	LoRA	QLoRA
Base model precision	FP16 / BF16	4-bit (NF4)
Trainable	low-rank adapters	low-rank adapters
Memory for 7B	∼24 GB	∼8 GB
Memory for 70B	out of reach for one GPU	∼24 GB
Quality vs full FT	nearly equivalent	nearly equivalent
Best for	laptops with mid GPUs, A100 nodes	free Colab T4, single 24 GB GPU

Default for AIMS labs

QLoRA + Unsloth on a free Colab T4. Works well with small open models such as Qwen3-0.6B, Qwen2.5-1.5B, or Gemma 2 2B when the architecture is supported.

Why Unsloth in This Course

What Unsloth gives you

- **2 to 5x faster** LoRA / QLoRA training,
- **80% less memory** during training,
- custom CUDA kernels for attention and MLP,
- **full Hugging Face compatibility**: works with Trainer, PEFT, TRL.

Practical consequence

We can fine-tune Qwen3-0.6B on a free **Colab T4 (16 GB)** during a short lab, with margin for evaluation runs.

Caveat

Unsloth optimizes a curated list of model architectures. Always check the support table for the exact model you target.

The Instruction-Response Format

Standard schema

Each training example is a JSON object with three fields: `instruction`, `input` (optional), and `output`.

Senegalese health example

```
{
  "instruction": "Reformule en wolof simple ce conseil medical.",
  "input": "Buvez beaucoup d'eau pendant les fortes chaleurs.",
  "output": "Naan ndox bu bari ci jamono ji tang yi."
}
```

AIMS academic example

```
{
  "instruction": "Resume cet extrait pour un etudiant Master 2 d'AIMS.",
  "input": "Self-attention computes a weighted sum of values...",
  "output": "Self-attention pondere les valeurs par la similarite query-cle."
}
```

Quality Over Quantity

Empirical lesson

1 000 high-quality examples typically beat 100 000 noisy ones for instruction-style fine-tuning. The LIMA paper showed this at scale.

Quality checklist

- answers are correct and consistent,
- style matches your target,
- instructions are diverse,
- no leakage from the eval set.

For African languages

Datasets often do not exist (Wolof, Pulaar, Bambara, Hausa, Swahili medical tone). Two options:

- collect a small seed set by hand,
- generate **synthetic data** with an LLM teacher.

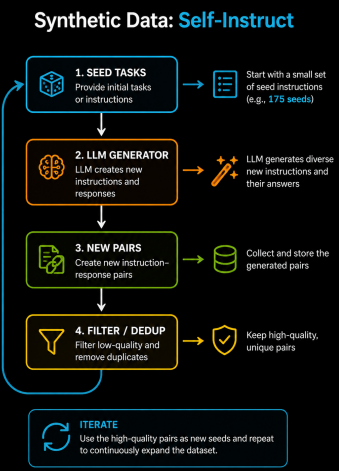
Synthetic Data: Self-Instruct

Idea (Wang et al., 2022)

Start with ∼175 **seed instructions** written by hand. Ask a strong LLM to generate new instructions and corresponding responses. Filter, deduplicate, repeat.

AIMS-Senegal seed sketch

- "Explique en francais simple le concept de ..."
- "Traduis cette phrase scientifique en wolof."
- "Reformule cet enonce de probleme pour un debutant."



Synthetic Data: Distillation from a Strong Teacher

Idea

Use a **larger / stronger model** (Qwen2.5-72B, Llama 3 70B, GPT-4, Claude) as a teacher to generate **instruction-response pairs**, then fine-tune a smaller student (0.6B-7B) on them.

Senegal example

Teacher: Qwen2.5-72B or Llama 3 70B prompted to act as an AIMS professor.
Student: Qwen3-0.6B fine-tuned on 5 000 of the teacher's QA pairs.
Result: a small, deployable assistant that mimics the AIMS tone.

Licenses and ethics

Check the teacher's license for synthetic data reuse. OpenAI and others restrict competing-model training; open-weight teachers (Qwen, Mistral, Llama) are usually more permissive, but always check the license.

Tools

Distilabel (Argilla / Hugging Face) automates prompt-response generation, scoring, and filtering pipelines.

Synthetic Data: Evol-Instruct and Quality Filtering

Evol-Instruct (Xu et al., 2023)

Take a base instruction and **progressively complexify** it: add constraints, deepen reasoning, broaden context. Used to build the WizardLM datasets.

Evolution example

1. `"Explique le palu."`
2. `"Explique le palu pour un agent de sante en Casamance."`
3. `"Explique le palu pour un agent de sante en Casamance, avec 3 conseils pratiques en wolof."`

Quality filtering

- **IFD score**: instruction-following difficulty; keep middle-range examples.
- **LLM-as-a-Judge**: prompt a strong model to score (instruction, response) pairs on relevance and quality.
- **Deduplication**: by hash and by semantic similarity.

Lab Goal: Fine-Tune a Small Model with Unsloth

What you will do

Take Qwen3-0.6B, fine-tune it with QLoRA on a small Senegal-flavored QA dataset, and compare the model **before** and **after** on the same prompts.

Steps

1. Install Unsloth, load the base model in 4-bit.
2. Attach LoRA adapters.
3. Prepare 200-500 instruction-response pairs.
4. Train for 1-2 epochs on Colab T4.
5. Save adapters, run inference, compare.

Suggested dataset themes

- AIMS academic FAQ in French,
- simple medical advice reformulated in Wolof,
- agriculture tips for Senegalese farmers,
- code explanations for Master 2 students.

Step 1: Install and Load (Unsloth, Colab T4)

```
!pip install -q unsloth peft trl datasets

from unsloth import FastLanguageModel

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "Qwen/Qwen3-0.6B",
    max_seq_length = 2048,
    dtype = None,            # auto: bfloat16 on Ampere+, float16 elsewhere
    load_in_4bit = True,     # this is what makes QLoRA cheap
)
print(model.get_memory_footprint() / 1e6, "MB")
```

What just happened

Unsloth loaded Qwen3-0.6B in 4-bit, attached its custom CUDA kernels, and gave you a frozen base model. The memory footprint stays low enough for a free Colab T4.

Step 2: Attach LoRA Adapters

```python
model = FastLanguageModel.get_peft_model(
    model,
    r = 16,
    lora_alpha = 32,
    lora_dropout = 0.05,
    target_modules = [
        "q_proj", "k_proj", "v_proj", "o_proj",  # attention
        "gate_proj", "up_proj", "down_proj",     # MLP
    ],
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 42,
)

trainable, total = model.get_nb_trainable_parameters()
print(f"trainable: {trainable:,}  total: {total:,}  ratio: {trainable/total:.2%}")
```

Expected output

Roughly 0.5 to 1% of the parameters become trainable. Everything else stays frozen.

Step 3: Prepare a Tiny Senegal-Flavored Dataset

```python
from datasets import Dataset

raw = [
    {"instruction": "Explique en 2 phrases le palu en wolof simple.",
     "output": "Sibbiru mooy feebar bu am ay yengu-yenguam ci xeet biy yenguy doom..."},
    {"instruction": "Resume ce paragraphe pour un etudiant AIMS.",
     "input":  "Self-attention computes a weighted sum...",
     "output": "Chaque token se compare aux autres et combine leurs informations..."},
    # ... 300 to 500 such pairs in your real dataset
]

def to_chat(ex):
    user = ex["instruction"] + ("\n\n" + ex["input"] if ex.get("input") else "")
    return {"text": tokenizer.apply_chat_template(
        [{"role":"user","content":user}, {"role":"assistant","content":ex["output"]}],
        tokenize=False)}

ds = Dataset.from_list(raw).map(to_chat)
```

Step 4: Train with TRL SFTTrainer

```python
from trl import SFTTrainer, SFTConfig

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=ds,
    dataset_text_field="text",
    args = SFTConfig(
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,
        num_train_epochs=2,
        learning_rate=2e-4,
        bf16=True,
        optim="adamw_8bit",
        output_dir="outputs",
    ),
)
trainer.train()
model.save_pretrained("aims_lora_adapters")
```

Expected wall time

A few hundred examples should complete quickly enough for an in-class demo on Colab T4.

Step 5: Compare Before vs After

```python
def gen(prompt):
    FastLanguageModel.for_inference(model)
    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
    out = model.generate(**inputs, max_new_tokens=200, do_sample=False)
    return tokenizer.decode(out[0], skip_special_tokens=True)

prompts = [
    "Explique en 2 phrases le palu en wolof simple.",
    "Resume cet enonce pour un etudiant AIMS: 'The convolution operates locally...'",
    "Donne 3 conseils en wolof pour un agriculteur en saison seche.",
]

# Run BEFORE training (re-load base) and AFTER (with adapters)
for p in prompts:
    print(p, "->", gen(p)[-300:])
```

What to look for

Look for tone shift, fewer unnecessary refusals, and more consistent format. Factual depth should stay similar because we did not add knowledge.

Advanced Extension: Hyperparameter Sweep

Goal

Vary LoRA hyperparameters and measure their effect on final loss and held-out quality.

Sweep grid

- rank $r \in \{4, 8, 16, 32\}$
- alpha $\in \{8, 16, 32, 64\}$ (often $\alpha = 2r$)
- dropout $\in \{0.0, 0.05, 0.1\}$
- target modules: attention only vs attention+MLP

What to expect

- bigger r helps until saturation,
- too high α destabilizes,
- dropout helps small datasets,
- attention+MLP usually beats attention-only.

Common Pitfalls

Catastrophic forgetting

Aggressive fine-tunes wipe useful general behavior. Mitigate with smaller learning rates and shorter training.

Train / inference mismatch

Use the **same chat template** at training and inference. A subtle change can collapse quality.

Eval set leakage

Hold out evaluation prompts **before** generating synthetic data.

Tokenizer drift

Some scripts (Wolof, Pulaar) tokenize poorly with English-centric tokenizers. Check token counts and BPE merges before training.

African Use Cases for LoRA / QLoRA

Health

- triage assistant fine-tuned on Ministry of Health protocols,
- Wolof reformulator for prescription advice.

Education

- AIMS academic tutor with consistent French academic tone,
- math problem solver for high-school Senegalese curriculum.

Agriculture

- ISRA-style advice in Wolof or Pulaar,
- structured weather alerts for ANACIM bulletins.

Public services

- DGID assistant fine-tuned to follow the official Senegalese tax-form vocabulary,
- ANSD assistant returning structured statistical answers.

What to Remember

1. Fine-tune to change **behavior**, not to add knowledge.
2. LoRA freezes W and learns a low-rank update BA, training $\sim$1% of parameters.
3. QLoRA = LoRA on a 4-bit base. It can make large open models trainable on a single accessible GPU.
4. Unsloth makes a small-model QLoRA fine-tune fit on a free Colab T4.
5. Quality of the dataset dominates. 1 000 clean examples beat 100 000 noisy ones.
6. For African languages: build seed sets, then expand with Self-Instruct, distillation, or Evol-Instruct, with quality filtering.

Key Resources

Core references

- Hu et al., *LoRA* (ICLR 2022)
- Dettmers et al., *QLoRA* (NeurIPS 2023)
- Practical LoRA Guide 2025 (Hugging Face)
- Unsloth GitHub and notebooks

Synthetic data

- Wang et al., *Self-Instruct*
- Xu et al., *WizardLM / Evol-Instruct*
- Synthetic Data Guide (Prem AI)
- Distilabel (Argilla / HF)

Bridge to Session 8: Evaluation and Project

Next step

You can now prompt, retrieve, and adapt. Session 8 closes the loop: how do we **measure** that any of this actually works, and how do we frame a final project that ships value?

Questions & Discussion

Fine-tune behavior. Retrieve knowledge. Prompt the rest.

LoRA and QLoRA are practical tools for adapting style, format, and domain behavior without retraining the full model.

Dr. Papa-Séga WADE

AIMS Senegal | GAAI-AIMS